

«Talento Tech»

React JS

Clase 04



Clase N° 4 | Flujo de datos, Hooks y Eventos

Índice

Flujo de datos, Hooks y Eventos

Flujo de datos: De contenedor a presentacional

Nivel 1: El Cerebro - ItemListContainer.jsx

Nivel 2: El organizador - ItemList.jsx

Nivel 3: El exhibidor - Item.jsx

Paso final: El ensamblaje en App.jsx y diseñar

Vista previa

Eventos: Haciendo que las cosas Reaccionen

Estados: Dándole memoria a los componentes con usestate

Componente contador

Componente Contador.jsx

Explicación paso a paso

Conceptos clave para llevarse

Estado + evento

Uniendo useState y Eventos en Nuestro Componente Item

La Magia del Estado Local

Tu primer tarea en TalentoLab

Objetivos de la Clase:

- Practicaremos el flujo de datos contenedor-presentacional
- Manejar eventos comunes en React, como clics.
- Aprender a gestionar el estado local de un componente usando el hook useState.
- Crear una funcionalidad básica para agregar productos a un carrito de compras.

Flujo de datos: De contenedor a presentacional

Ahora vamos a aplicar el patrón que ya conocemos, pero en una estructura de tres niveles, algo súper común en aplicaciones reales. Queremos mostrar una lista de productos en nuestra tienda. Para lograrlo de forma ordenada y escalable, vamos a repartir el trabajo en tres componentes especializados, cada uno con una misión clara: Dentro de nuestra carpeta Componentes, vamos a crear los componentes que vemos entre paréntesis.

1. **El Cerebro (ItemListContainer):** Es el componente **contenedor**. Su única misión es **tener la lógica y conseguir los datos**. No le preocupa en lo más mínimo cómo se van a ver esos datos.
2. **El Organizador (ItemList):** Este componente es un intermediario. Recibe la lista completa de datos desde "El Cerebro" y su trabajo es **organizar y preparar** esa información. Se encarga de recorrer la lista y delegar la tarea de mostrar cada ítem individual.
3. **El Exhibidor (Item):** Este es el componente **presentacional** puro. Es el encargado de la apariencia. No sabe de dónde vienen los datos, ni le importa. Solo recibe la información de **un único producto** y se concentra en mostrarlo de la forma más prolija y atractiva posible.

Nivel 1: El Cerebro - ItemListContainer.jsx

Este es nuestro punto de partida. Como componente contenedor, su responsabilidad es manejar los datos. En una aplicación real, acá haríamos una llamada a una base de datos o una API para buscar los productos. Para nuestra clase, vamos a simularlo creando un array directamente en el código.

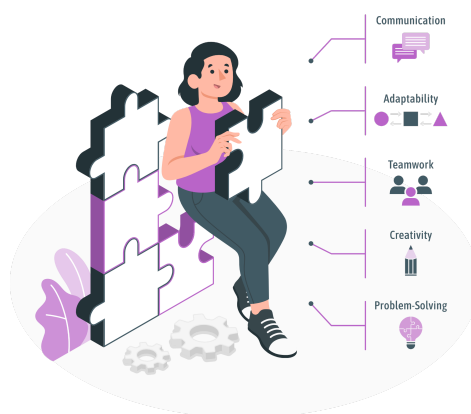
```
// En /componentes/ItemListContainer/ItemListContainer.jsx
import { ItemList } from "../ItemList/ItemList";
export function ItemListContainer({ Mensaje }) {
  const productos = [
    { id: '1234', nombre: 'Notebook Pro', precio: 12000, stock: 15 },
    { id: '2344', nombre: 'Monitor Curvo', precio: 45000, stock: 25 },
    { id: '2545', nombre: 'Teclado Mecánico', precio: 15000, stock: 50 },
  ];
  return (
    <div>
      <h2>{Mensaje}</h2>
      <div>
        <ItemList productos={productos} />
      </div>
    </div>
  );
}
```

En resumen: ItemListContainer imprime un mensaje en h2 y obtiene los datos que luego se los pasa a su hijo, ItemList. No hace más.

Nivel 2: El organizador - ItemList.jsx

Este componente recibe la lista. No le importa cómo se ve cada producto, solo se encarga de **recorrer el array (.map())** y decirle al siguiente nivel que renderice cada uno. ItemList recibe una lista y la transforma en múltiples componentes Item, pasándole a cada uno los datos que le corresponden.

Prestá atención a la forma de importación. Escribi import y busca entre las opciones el componente que quieras importar.



```
// En /components/ItemList/ItemList.jsx
import { Item } from "../Item/Item";

export function ItemList({ productos }) {
  return (
    <div style={{ display: 'flex', gap: '20px' }}>
      {productos.map(prod => (
        <Item key={prod.id} {...prod} />
      ))}
    </div>
  );
}
```

Detalles del código:

1. Con el div agrupamos los elementos y le damos estilo en línea a este grupo puntualmente.
2. Con el mapeo, lo que estamos haciendo es lo siguiente: para cada "prod" en el array "productos", creamos un componente "Item", al que le pasamos una "key" única. (Muy importante para optimizar)
3. Usamos el spread operator {...prod} para pasar todas las propiedades del objeto "prod" (id, nombre, precio, stock) como props individuales al componente Item

? 🧠 ¿Cómo harías para mapear los productos y mostrar solamente los que tengas stock?

Nivel 3: El exhibidor - Item.jsx

Este es nuestro componente **presentacional** o "dumb". No sabe de listas ni de lógica. Solo recibe las props de **un producto** y se encarga de mostrarlo de la forma más linda posible, usando sus propios estilos de CSS Modules.



```
// En /componentes/Item/Item.jsx

// Recibe las props usando destructuring
export function Item({ nombre, precio, stock }) {
  return (
    <div>
      <h3>{nombre}</h3>
      <p>Precio: ${precio}</p>
      <p>Stock disponible: {stock}</p>
      <button>Comprar</button>
    </div>
  );
}
```

Como quedaría entonces el flujo de nuestro código: Container tiene los datos -> se los pasa a List -> List mapea y le pasa los datos de cada uno a Item -> Item los muestra. ¡Perfecto!

Paso final: El ensamblaje en App.jsx y diseñar

Ensamblado

Ya tenemos toda nuestra cadena de componentes lista. Ahora, para que todo esto se vea en nuestra página, necesitamos "llamar" al componente de principal de nuestra cadena, el ItemListContainer, desde nuestro componente principal, que es App.jsx.

App.jsx es el centro de nuestra aplicación. Desde acá orquestamos qué componentes se muestran en la página.

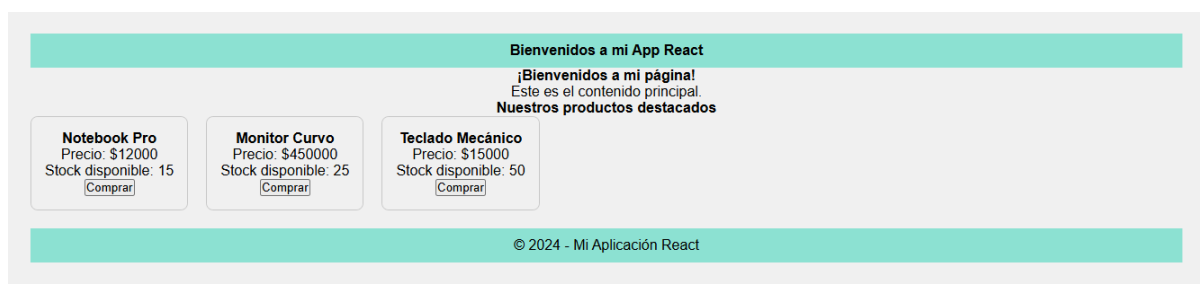
Ver imagen en la siguiente página

```
// /src/App.jsx
import './App.css';
import { Layout } from './componentes/layout/Layout';
import { ItemListContainer } from
'./componentes/ItemListContainer/ItemListContainer';

function App() {
  return (
    <Layout>
      { /* Todo lo que pongamos acá adentro irá donde estaba {children} */ }
      <h1>¡Bienvenidos a mi página!</h1>
      <p>Este es el contenido principal.</p>
      <ItemListContainer Mensaje="Nuestros productos destacados" />
    </Layout>
  );
}

export default App;
```

Una vez que tenemos todo organizado en nuestro archivo App.jsx, (si seguiste el paso a paso) vamos a ver algo similar a esto:



Diseño

Para que nuestro componente quede mas lindo, vamos a aplicarle CSS modules. Para eso, nos vamos a nuestra carpeta del componente, y creamos el modulo par ese componente. Recordá que es el mismo nombre que el archivo .jsx con la extensión module.css.

Voy a usar flex para centrar los productos y padding para distanciarlos

```

/* En /componentes/ItemListContainer/ItemListContainer.module.css*/
.subtitulo{
  padding:1rem 1rem 0 1rem;
}
.productos{
  display: flex;
  flex-direction: row;
  justify-content: center;
  align-items: center;

  padding:2rem;
}
    
```

Vista previa

Nuestra aplicación se verá de la siguiente manera:



Eventos: Haciendo que las cosas Reaccionen



Vemos que esteticamente nuestra página va quedando genial, pero ese botón de "Comprar" no hace nada, está de adorno. Vamos a comenzar a darle vida. Para eso, necesitamos manejar **eventos**.

Un evento es cualquier cosa que hace el usuario: un clic, pasar el mouse por encima de algo, escribir en un campo de texto, etc. React nos facilita muchísimo escuchar y responder a estos eventos.

La sintaxis es muy parecida a HTML pero en camelCase. En lugar de onclick, usamos onClick.

Vamos a modificar nuestro componente Item.jsx para que, cuando hagamos clic en el botón, nos muestre una alerta. Para eso creamos una función flecha que nos permita enviar un alert con un mensaje en particular.

Si quiero que mi mensaje, contenga un dato en particular se lo agrego de esta manera: **`${dato A Mostrar}`**.

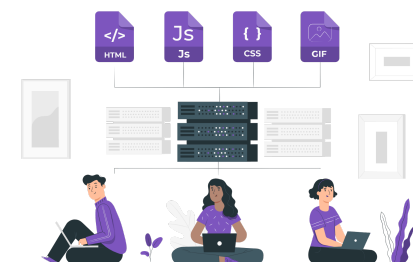
```
// En /componentes/Item/Item.jsx
export function Item({ nombre, precio, stock }) {
  const CompraClick = () => { // Quiero que se ejecute cuando le doy clic
    alert(`¡Agregaste ${nombre} al chango!`);
  };
  return (
    <div style={{ border: '1px solid #ccc', padding: '15px', borderRadius: '8px' }}>
      <h3>{nombre}</h3>
      <p>Precio: ${precio}</p>
      <p>Stock disponible: {stock}</p>
      <button onClick={CompraClick}>Comprar</button>
    </div>
  );
}
```

Ahora nuestro botón es interactivo y pudimos hacer que nuestra aplicación responda al usuario. Otros eventos comunes son: **onChange**, detecta cambios en los inputs, **onSubmit**, se dispara cuando el formulario es enviado, **onMouseOver**, **onMouseOut** para manejar los eventos del mouse.

Estados: Dándole memoria a los componentes con usestate

La alerta está buena, pero no es muy útil. Nosotros queremos poder llevar la cuenta de cuántos productos queremos. Para eso, nuestros componentes necesitan **memoria**. Necesitan poder *recordar* cosas que van cambiando. En React, esa memoria se llama **estado**.

Para manejar el estado en componentes funcionales, React nos da unas herramientas muy buenas llamadas **Hooks**. El primer hook que vamos a ver es useState.



¿Cómo funciona useState?

useState es una función que nos devuelve un array con dos cosas:

1. **El valor actual del estado:** La "cosa" que queremos recordar (un número, un texto, una lista, un bool, etc.).
2. **Una función para actualizar ese valor:** La única forma correcta de cambiar el estado.

Vamos a verlo con un ejemplo súper clásico y claro: un contador.

Componente contador

Vamos a nuestra carpeta de componentes y creamos una carpeta llamada Contador. Dentro generamos su componente.

Componente Contador.jsx

```
// En componentes/Contador/Contador.jsx

// Importamos 'useState' desde React.
import { useState } from 'react';

export function Contador() {
  const [contador, setContador] = useState(0);
  const incrementar = () => {
    setContador(contador + 1); //¡Usamos la función para actualizar el estado!
  };
  //Continúa!
```

```
const decrementar = () => {
  setContador(contador - 1);
};

return (
  <div style={{ margin: '20px', padding: '20px', border: '1px solid black' }}>
    <h3>Contador de Ejemplo</h3>
    <p>Valor actual: {contador}</p>
    <button onClick={incrementar}>Sumar +1</button>
    <button onClick={decrementar}>Restar -1</button>
  </div>
);
}
```

Explicación paso a paso

Importación de useState

```
import { useState } from 'react';
```

- Como importamos algún componente, desde React importamos el hook useState y cualquier otro.

Declaración del estado

Creamos una variable que desarma el return del useState. Recordemos que dijimos que devolvía un valor y una función.

```
const [contador, setContador] = useState(0);
```

- **contador:** valor actual del estado.
- **setContador:** función que actualiza ese valor.
- **useState(0):** inicializa el contador en 0.

Clave: cada vez que llamamos a setContador, React vuelve a renderizar el componente con el nuevo valor.

Por convención, la función del hook se va a llamar set + “el nombre del valor actual del estado”

Funciones de actualización

Creamos dos funciones que van a modificar el valor del estado actual.

```
const incrementar = () => {setContador(contador + 1)};
const decrementar = () => {setContador(contador - 1)};
```

- incrementar: suma uno a la variable contador.
- decrementar: resta uno a la variable contador.
- **Nunca modificamos la variable contador directamente, siempre usamos setContador.**

Renderizado (lo que se ve en pantalla)

```
return (
  <div style={{ margin: '20px', padding: '20px', border: '1px solid black' }}>
    <h3>Contador de Ejemplo</h3>
    <p>Valor actual: {contador}</p>
    <button onClick={incrementar}>Sumar +1</button>
    <button onClick={decrementar}>Restar -1</button>
  </div>
);
```

- Se muestra el valor actual del contador.
- Dos botones permiten al usuario cambiar el valor actual del contador. Con cada clic disparamos un evento que llama a las funciones generadas: incrementar y decrementar

Flujo de funcionamiento

1. El componente se inicia → contador comienza en 0.
2. El usuario hace clic en “Sumar +1” → se ejecuta incrementar.
3. setContador actualiza el valor → React vuelve a renderizar.
4. En pantalla aparece el nuevo valor.
5. El mismo ciclo ocurre si se presiona “Restar -1”.

Conceptos clave para llevarse

- useState = Hook para manejar estado en componentes.
- setContador = única forma válida de actualizar el estado.
- Cada cambio en el estado provoca un re-render del componente.
- Estado = memoria interna del componente que cambia con el tiempo.

Estado + evento

Vamos a fusionar los dos conceptos: el **evento** del clic y el **estado** de useState, pero dentro de nuestro componente `Item.jsx`. La idea es que *cada* producto en nuestra vidriera tenga su propio contador **independiente**.

Uniando useState y Eventos en Nuestro Componente Item

Vamos a modificar el archivo `Item.jsx` para que no solo muestre la información del producto, sino que también nos permita elegir cuántas unidades queremos agregar a un futuro carrito de compras.



Intentá agregar el `useState` al componente `Item` y luego pasa a ver la solución 😊

Así es como quedaría el componente `Item.jsx`

```
// En /componentes/Item/Item.jsx
import { useState } from 'react';
export function Item({ id, nombre, precio, stock }) {
  const [cantidad, setCantidad] = useState(0);
  const incrementar = () => {
    if (cantidad < stock) {
      setCantidad(cantidad + 1);
    }
  };

  const decrementar = () => {
    if (cantidad > 1) {
      setCantidad(cantidad - 1);
    }
  };

  const agregarAlCarrito = () => {
    alert(`Agregaste ${cantidad} unidades de ${nombre} al carrito.`);
  }

  return (
    <div style={{ border: '1px solid #ccc', padding: '15px', borderRadius: '8px', textAlign: 'center' }}>
      <h3>{nombre}</h3>
      <p>Precio: ${precio}</p>
      <p>Stock disponible: {stock}</p>
    </div>
  );
}
```

```

        <div style={{ display: 'flex', alignItems: 'center', justifyContent:
        'center', margin: '10px 0' }}>
          <button onClick={decrementar}>-</button>
          <p style={{ margin: '0 10px' }}>{cantidad}</p>
          <button onClick={incrementar}>+</button>
        </div>

        <button onClick={agregarAlCarrito}>Agregar al Carrito</button>
      </div>
    );
  }
}
    
```

¿Qué hicimos acá? Repasemos, paso a paso:



1. **Importamos useState:** Si no lo importamos, React no sabe de qué le estamos hablando.

2. **Creamos el estado cantidad:** Dentro de la función Item, llamamos a useState(0). Le decimos: "Che, React, para este componente Item en particular, guardate un valor que se va a llamar cantidad. Arrancá en 0".

3. **Creamos las funciones incrementar y decrementar:** Igual que anteriormente, van a llamar a setCantidad. Fíjate que le pusimos una pequeña lógica para que no puedas pedir más productos que el stock disponible, ni menos de uno.
¡Pequeños detalles que suman!

4. **Creamos la función agregarAlCarrito:** Este es el reemplazo de nuestro CompraClick de antes. Ahora es más inteligente: no solo sabe *qué* producto es (nombre), sino también *cuántos* (cantidad).

5. **Unimos todo en el return:**

- Mostramos la cantidad actual en un párrafo.
- A los botones de - y + les asignamos su función correspondiente en el evento onClick.
- Al botón principal le asignamos la función agregarAlCarrito.

La Magia del Estado Local

Ahora, cuando vayas a tu aplicación, vas a ver que cada producto tiene su propio contador. Podés poner 3 notebooks, 1 monitor y 5 teclados, y **cada componente Item recuerda su propia cantidad sin mezclarse con los demás**.

Esto es el **estado local**. Es una "memoria" que vive y muere dentro de un componente específico. Es una de las ideas más potentes de React y la acabamos de implementar.

Con todo lo que hicimos hoy, nuestra página se verá de la siguiente manera:



Reflexión final

Aprendimos dos conceptos clave en React: la gestión del estado local con **useState** y el manejo de eventos, ambos esenciales para crear aplicaciones web dinámicas e interactivas. Durante la práctica, aplicamos estos conocimientos para construir una funcionalidad de carrito de compras, simulando una tarea real en el desarrollo de un eCommerce. Esto no solo refuerza los conceptos teóricos, sino que también prepara a los estudiantes para resolver problemas similares en proyectos más complejos.

La gestión del estado nos permitió almacenar y manipular datos dinámicos, mientras que el manejo de eventos nos ayudó a interactuar con el usuario de manera efectiva. Este enfoque modular y práctico fortalece las bases necesarias para avanzar hacia aplicaciones más sofisticadas.

Tu primer tarea en TalentoLab

¡Felicitaciones! 🎉 Has sido seleccionado para formar parte de TalentoLab, una empresa innovadora que desarrolla soluciones tecnológicas. Serás parte del equipo responsable de crear un eCommerce para un cliente.



Tu tarea inicial será crear una funcionalidad básica para el manejo de un carrito de compras.

Marcar productos como favoritos

El Objetivo de este desafío es que pongas en práctica el useState para cambiar el estado de otra funcionalidad muy importante de nuestra página: Los favoritos. Al lado del boton de agregar producto, vamos a mostrar una estrella. Si le hacés clic, la estrella se "rellena" (cambia de apariencia) para indicar que es un favorito. Si volvés a hacer clic, vuelve a su estado original. Cada producto maneja su propio estado de favorito.

Pasos a seguir:

1. Crea el estado con su función y su variable.
2. Genera la función marcarComoFavorito que cambie el valor actual de un booleano (que si está en True, al llamar a la función pase a False y viceversa)
3. Asignale la función a una etiqueta en particular, en la posición seleccionada.

Una forma en la que puede quedar nuestra salida con su función asociada a un estado, sería así:

```

<span onClick={marcarComoFavorito}
  style={{ fontSize: '24px'}}
  > {esFavorito ? '★' : '☆'}
</span>
    
```

Material y Recursos Adicionales:

- ★ [Documentación oficial de React - useState](#)
 - ★ [React Events Handling - Documentación](#)
 - ★ [Ejercicios prácticos sobre React Hooks](#)
-

Preguntas para Reflexionar:

- ¿Qué beneficios ofrece React al manejar eventos comparado con JavaScript puro?
 - ¿Qué aprendiste al trabajar con el flujo contenedor-presentacional en React? ¿Qué desafíos enfrentaste?
 - ¿Cómo podrías extender la aplicación para incluir funcionalidades como eliminar productos individuales del carrito?
-

Próximos Pasos:

- Comprender qué son los efectos secundarios y cómo gestionarlos con el hook **useEffect**.
- Realizar peticiones a APIs para obtener datos dinámicamente.
- Manejar estados de carga y errores para mejorar la experiencia del usuario.



Buenos Aires
aprende
Agencia de Políticas para el Futuro

BA Buenos
Aires
Ciudad